Binary classification (yes or no) and how to measure it

Can a binary classifier say yes or no to "high cooling load", how does it learn, and how do we count and price its mistakes?

CML3112 | Civil Engineering and Architecture | Week 9, class 2 of 4 | Tuesday 10 November 2026, 11:00 to 14:00
KLAi | International University of East Africa | Jotham Martin Wambi

Today in one view: what you will leave with

By the end of today you can



Turn the cooling load into a binary label with a limit, and standardise the features with the mean and standard deviation



Train logistic regression and a decision tree classifier, and explain how logistic regression learns: log loss and gradient descent



Count TP, FN, FP and TN, work out precision, recall and F1, and choose a decision threshold with a cost table

Your three hours: ideas first, then the lab, then the quiz

0:00	10	Recall from Class 1
0:10	40	Concepts 1: labels, spread, how a classifier learns
0:50	35	Concepts 2: counting and pricing mistakes
1:25	15	Break
1:40	60	Lab: binary classification, in the notebook
2:40	15	Quiz opens: forms Q7 and Q8, 24 hours
2:55	5	Close and what is assessed

Concepts first, with no laptops. Then the lab, then the quiz.

Three questions from last time

1. What is a baseline model for?

2. What does the unit of the MAE tell you?

3. Why do we split the rows by equipment group?



Answer aloud first. Then we check together.

Answers to the three questions from Class 1

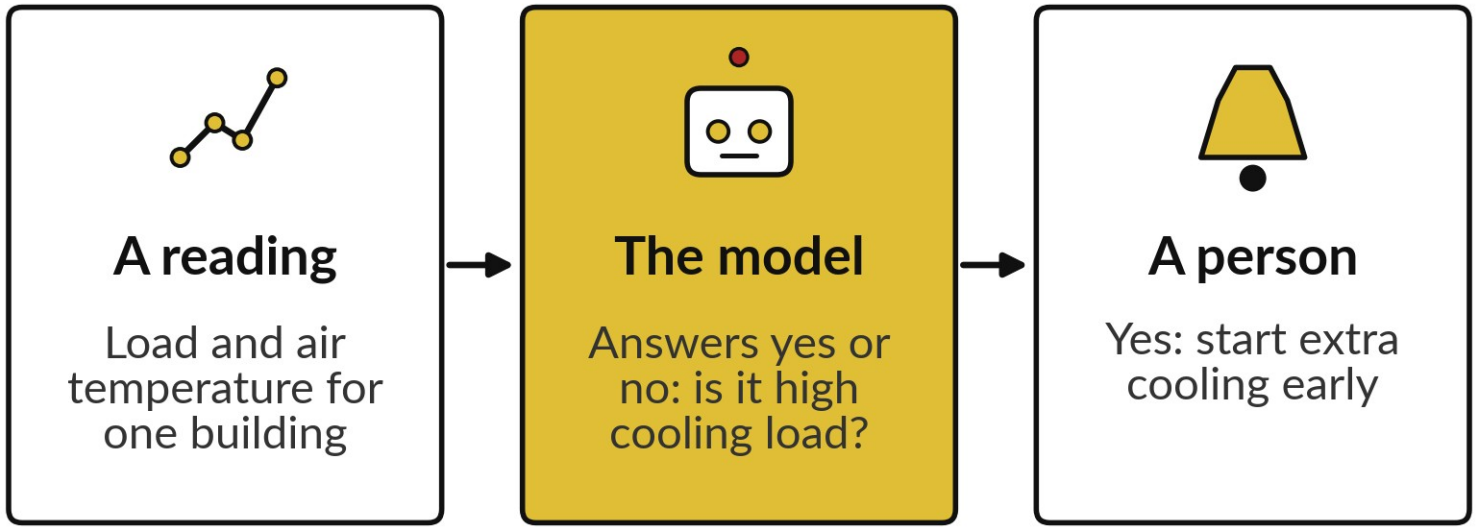
Question	Short answer
What is a baseline model for?	A baseline model is the simplest possible prediction, here the mean of the training target. Every model must beat it. If a model cannot, it has learned nothing useful.
What does the unit of the MAE tell you?	The mean absolute error (MAE) is in the unit of the target variable, so an MAE of 2 kW means the prediction error is about 2 kW on average.
Why do we split the rows by equipment group?	So that the validation set holds whole building groups the model has never met, which is the situation in real use.

Check your own answer against these before we start today.

Words we use today

Word	What it means
Binary classification	Two possible answers: the positive class (yes, such as "overheating") and the negative class (no). The label is 1 or 0.
Decision threshold	The probability at or above which the model predicts positive (yes).
Confusion matrix	A table of actual against predicted classes: TP, FN, FP and TN (true or false, positive or negative).
Precision and recall	Precision: of the predicted positives, the share that are right. Recall (sensitivity): of the actual positives, the share found.
ROC curve and AUC	The ROC curve plots true positive rate against false positive rate. AUC, the area under it, is 0 to 1: how well positives rank above negatives.

Binary classification gives a yes or no answer that tells a plant operator whether to act

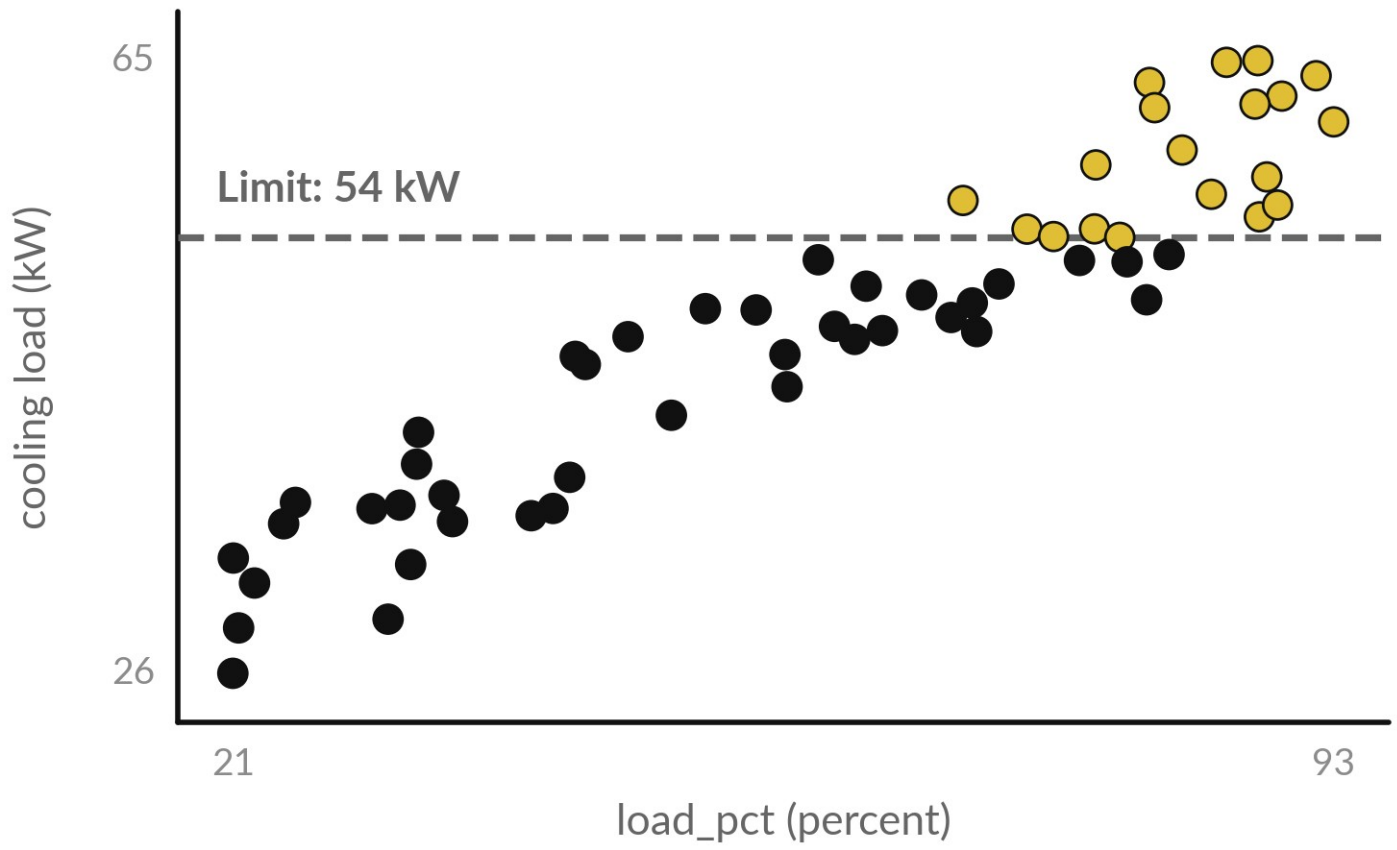


Binary classification has two classes: yes (positive) and no (negative). The question: is it high cooling load?

In Class 1 (regression) the model gave a number, such as the cooling load in kW, scored by MAE.

Today it gives one of two classes, and we count its right and wrong answers.

A limit on the cooling load turns each row into a positive or negative label



The label (target variable) is positive (1, yes) when the cooling load is 54 kW or more, and negative (0, no) below.

Each dot is one row of the training set. Gold dots are positive: 19 of 58.

Validation set: 3 of 15 rows are positive.

The features are still load_pct and ambient_c.

The majority-class baseline always answers no, is right on 80 percent of rows and finds no actual positive

Actual classes: 12 no, 3 yes



Majority-class baseline: always says "no"



Right on 12 of 15 rows. Accuracy 0.80.

Red: the 3 actual positives this baseline never finds.

A high accuracy that finds no positive is useless.

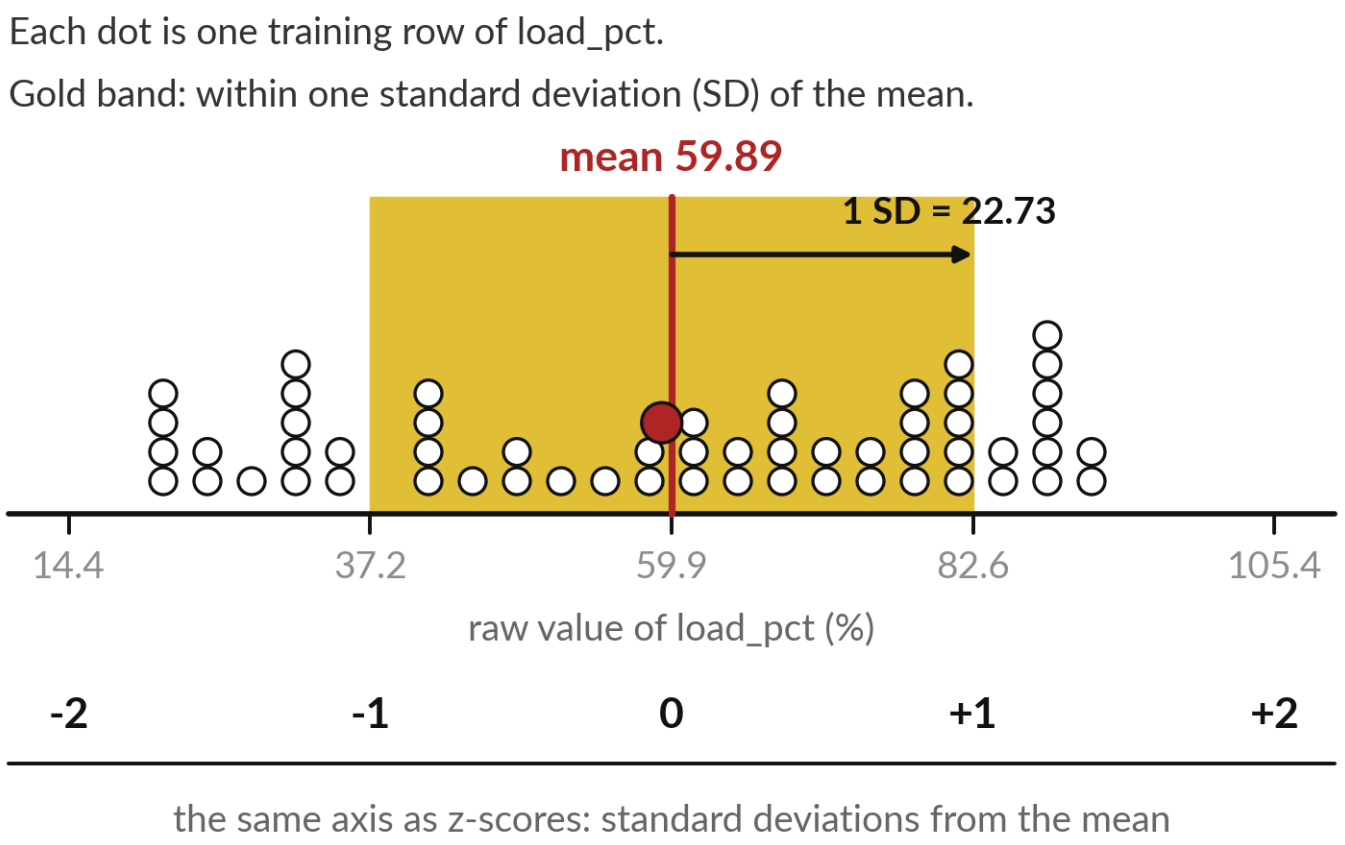
The majority-class baseline always predicts the more common class, here no. Accuracy is the share of rows answered correctly.

Always no is right on 12 of 15 validation rows: accuracy 0.80.

It finds none of the 3 actual positive rows.

A model must beat accuracy 0.80 and also find positive rows.

The standard deviation measures how far the values of a column typically sit from their mean



Standard deviation (SD): the typical distance of a value from its mean. StandardScaler rewrites each feature as z-scores.

load_pct: mean 59.89 %, SD 22.73 %. 33 of 58 rows lie in the gold band.

A z-score counts SDs above (+) or below (-) the mean. Red dot: training row 1.

The standard deviation and the z-score as formulas, with training row 1 worked by hand

Standard deviation

$$\sigma = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2}$$

Standardisation (z-score)

$$x' = (x - \mu) / \sigma$$

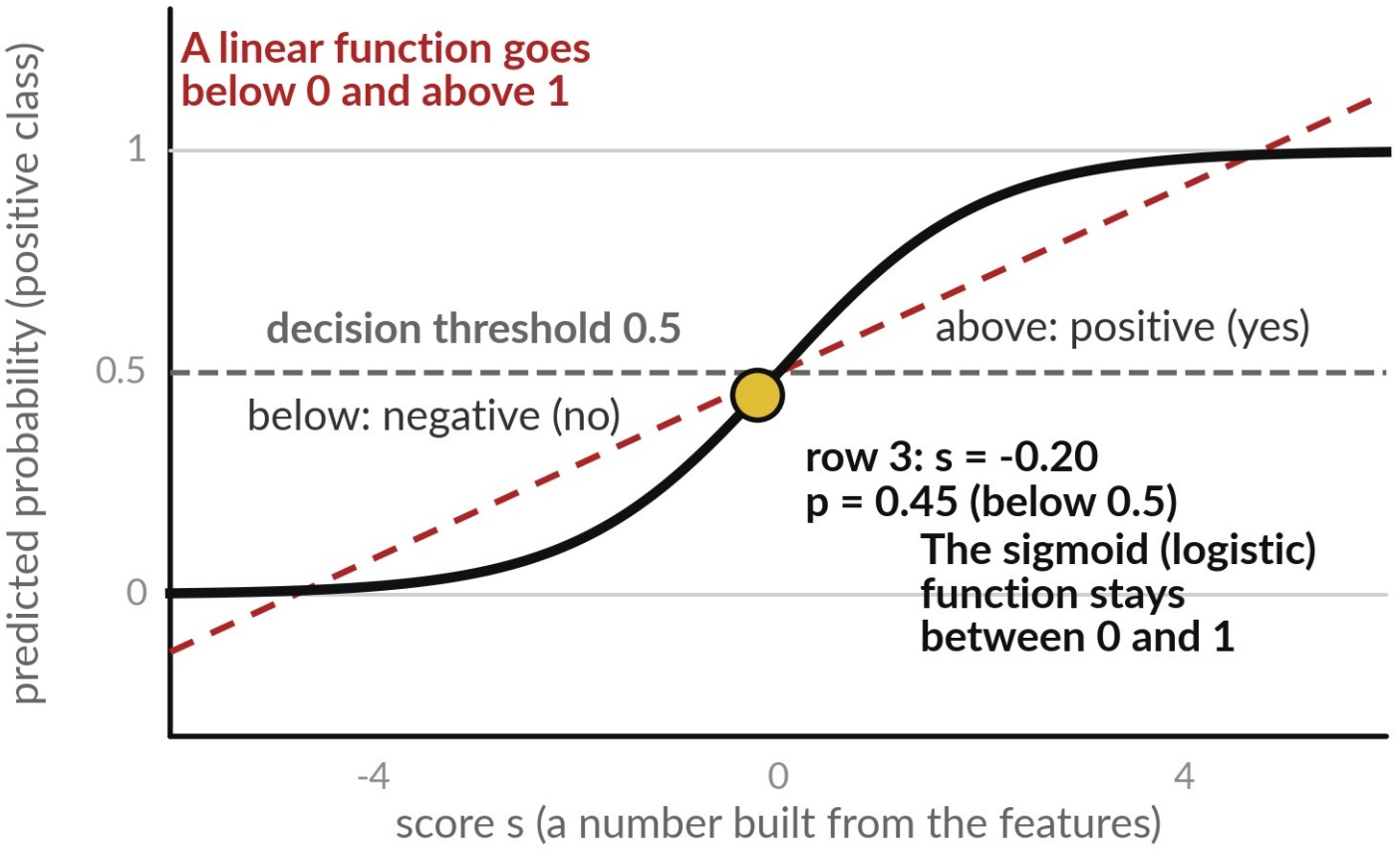
Training row 1, load_pct

$$x'_1 = (59.15 - 59.89)/22.73 = -0.03$$

Symbol	What it means	In our data
x_i, n	A raw value of row i, and the number of training rows	load_pct of the 58 training rows
mu	The training mean of that column	load 59.89 %; ambient 26.33 deg C
sigma	The training standard deviation of that column	load 22.73 %; ambient 4.55 deg C
x'	The z-score: 0 is the mean, 1 is one SD above	Training row 1: load -0.03, ambient 1.18

StandardScaler divides by n. pandas .std() divides by n - 1 and gives 22.93 for load_pct.

Logistic regression passes a linear score through the sigmoid (logistic) function, which stays between 0 and 1



The sigmoid (logistic) function turns any score s into a probability from 0 to 1.

A probability of 0.5 means the two classes are equally likely.

The decision threshold is the probability at or above which the model predicts positive (yes), and below which it predicts negative (no).

Logistic regression in three lines: score, probability, prediction

1 Score

$$s = b_0 + b_1 x_1' + b_2 x_2'$$

2 Probability

$$p = 1 / (1 + e^{-s})$$

3 Prediction

$$\hat{y} = \text{positive if } p \geq t, \text{ otherwise negative}$$

Symbol	What it means	In our data
s	The score: any number	Validation row 3 (record_id 17): s = -0.20
b0; b1, b2	Intercept (bias); weights (coefficients): rise in s per standard deviation	-1.71; 2.79 (load), 0.73 (ambient)
x1', x2'	Standardised load_pct and ambient_c (z-scores)	Validation row 3: 0.08 and 1.75
p	Probability of the positive class. e = 2.718	Validation row 3: p = 0.45
t	Decision threshold: predict positive if p is at least t	0.5 first, then 0.3 and 0.7
y-hat	The predicted class: positive (1) or negative (0)	Validation row 3: no

Worked example: validation row 3 gets p = 0.45, so the model predicts no

	A	B	C	D
	Part	**z-score x'**	**Weight b**	**Product**
1	Intercept b0			-1.71
2	load_pct	0.08	2.79	0.23
3	ambient_c	1.75	0.73	1.28
4	Score s (sum)			-0.20
5	Probability p			0.45

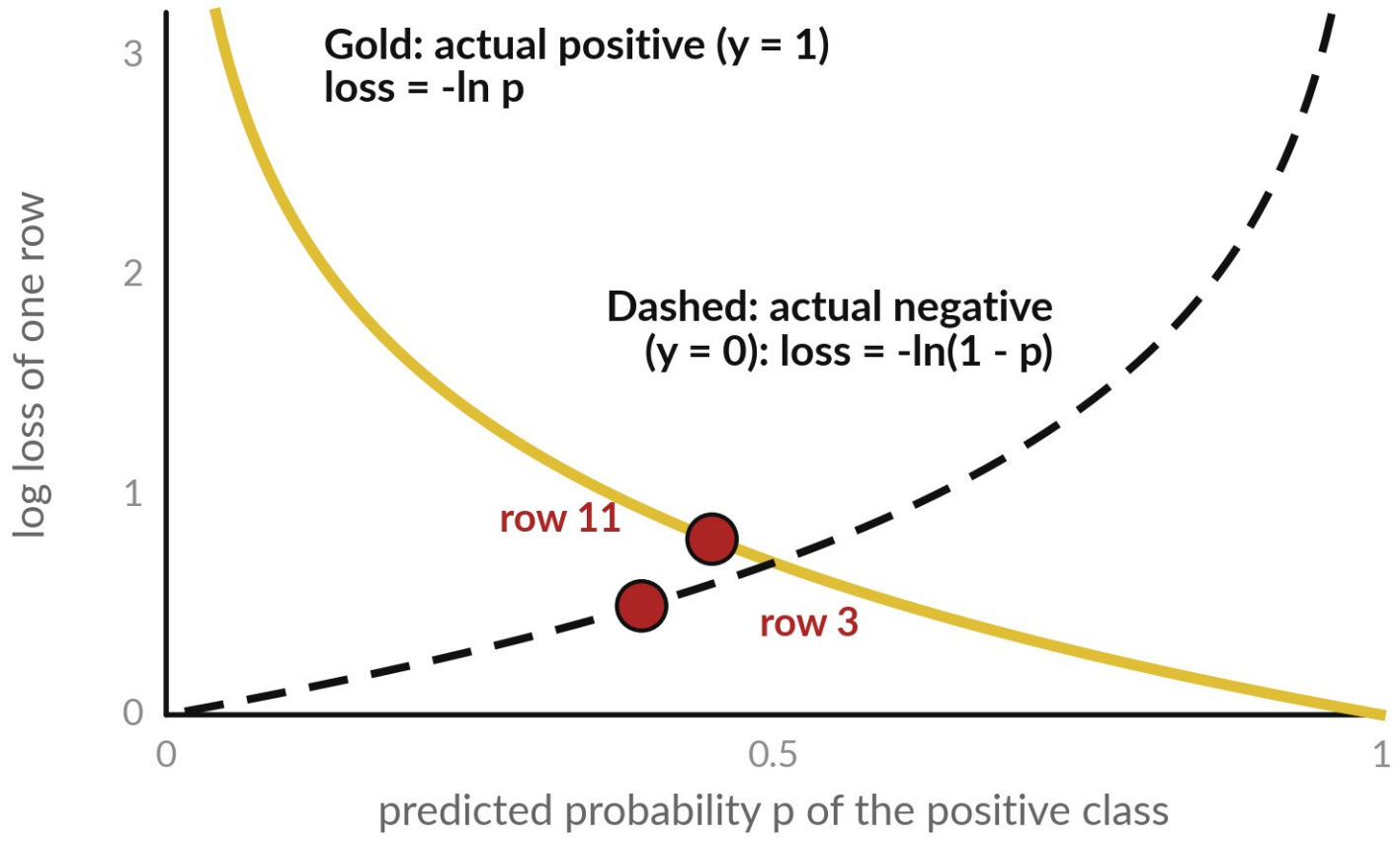
Validation row 3 (record_id 17): load_pct 61.76 %, ambient_c 34.28 deg C. Actual class: positive.

p = 0.45 is below 0.5: a false negative (FN).

One SD of load_pct (22.73 %) adds 2.79 to s: 0.12 per percent.

One SD of ambient_c (4.55 deg C) adds 0.73: 0.16 per deg C.

Log loss (binary cross-entropy) measures how wrong one predicted probability is



Log loss compares the probability p with the actual class y (1 or 0). It is 0 for a certain, right answer and grows fast as p moves away.

$-(y \ln p + (1 - y) \ln(1 - p))$

Row 3, actual positive: -ln 0.45 = 0.80.

Row 11, actual negative: -ln(1 - 0.39) = 0.49.

The training loss is the mean log loss, and gradient descent walks the weights downhill

Training loss

$$L = \frac{1}{n} \sum_{i=1}^{n} \text{loss}_i$$

Slope for weight j

$$\text{slope}_j = \frac{1}{n} \sum_{i=1}^{n} (p_i - y_i)\, x_{ij}$$

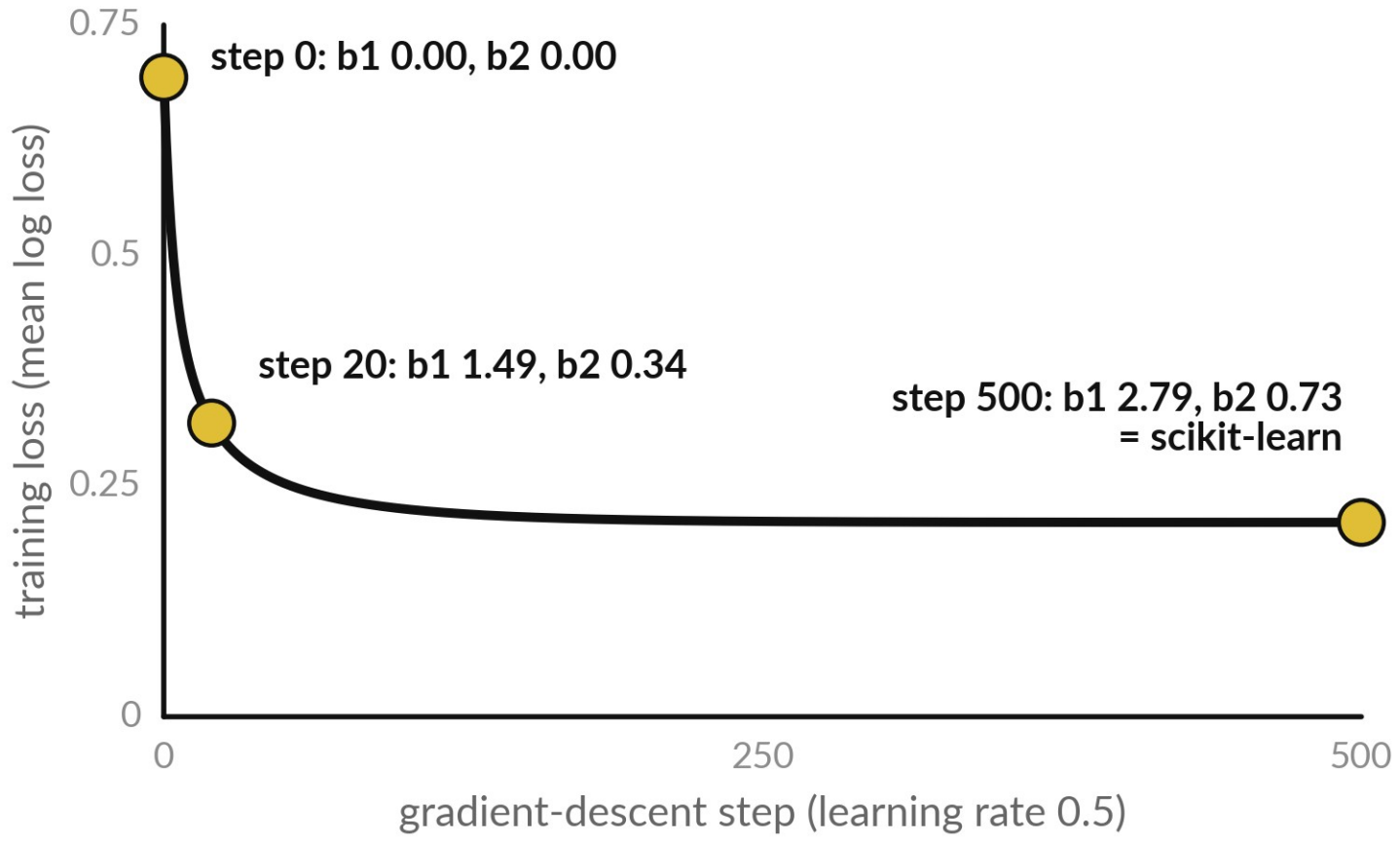
Update rule (Class 1)

$$b_j \leftarrow b_j - \eta \cdot \text{slope}_j$$

Symbol	What it means	In our data
L, n	Mean log loss over the n training rows	n = 58; L = 0.21 after training
p_i - y_i	Error of row i: probability minus actual class	Start: all weights 0, so every p_i = 0.5
x_ij	Feature j of row i (z-score); x_i0 = 1 for b0	j = 1 load_pct, j = 2 ambient_c
slope_j	Positive slope: lower b_j	Start: 0.17, -0.34, -0.04

eta is the learning rate (step size), 0.5 here. Start by hand: slope_0 = 0.5 - 19/58 = 0.17.

scikit-learn minimises the same log loss with a faster solver and a small L2 penalty that keeps the weights small



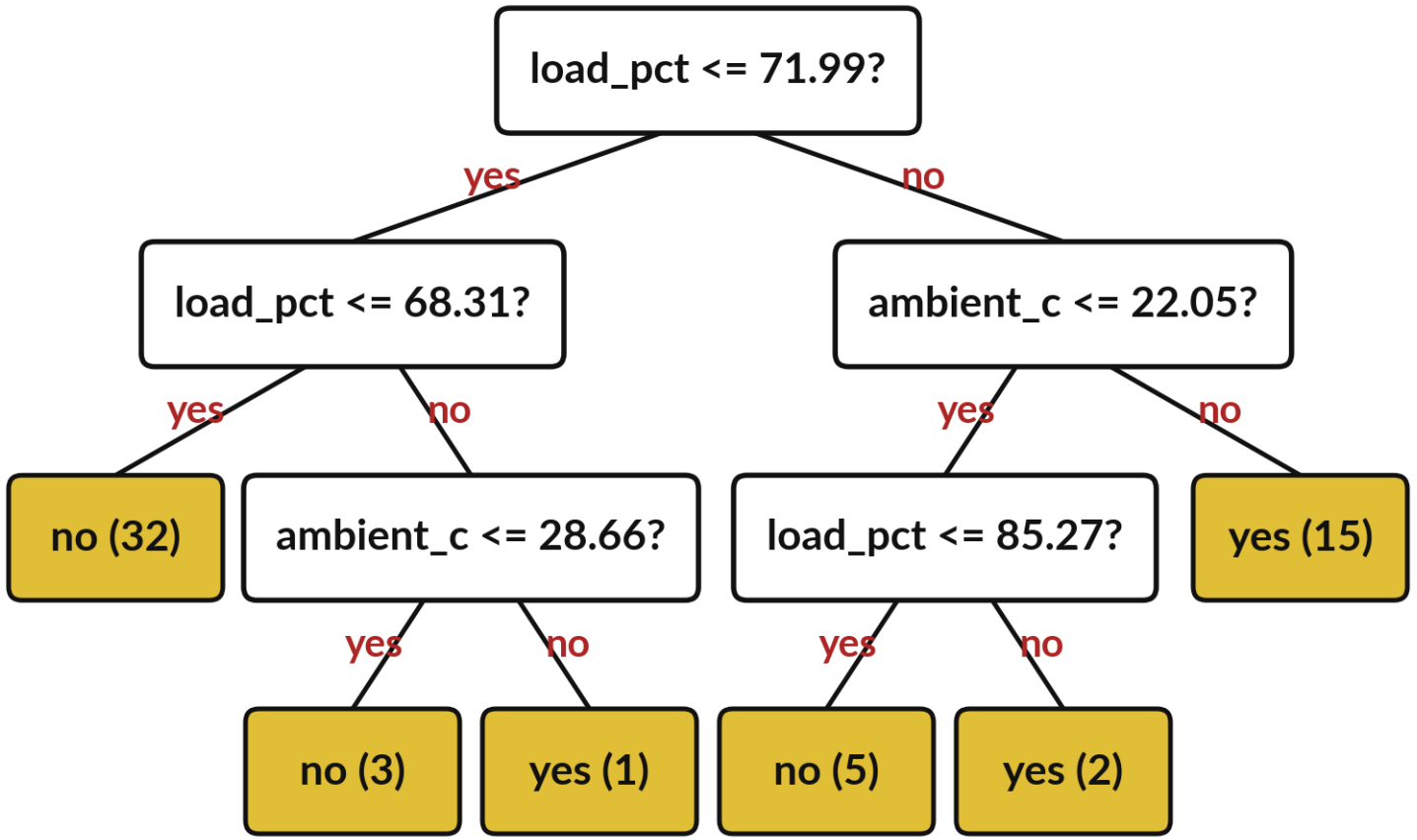
LogisticRegression() minimises C x (total log loss) + 0.5 x ($b1^2$ + $b2^2$). This L2 penalty keeps the weights small. C = 1.0.

Gradient descent with the same penalty reaches the same weights: 2.79 and 0.73.

The lbfgs solver gets there in a few smarter steps.

The penalty works in raw units, so standardise first.

A decision tree classifier can also answer yes or no, and it does not need standardisation



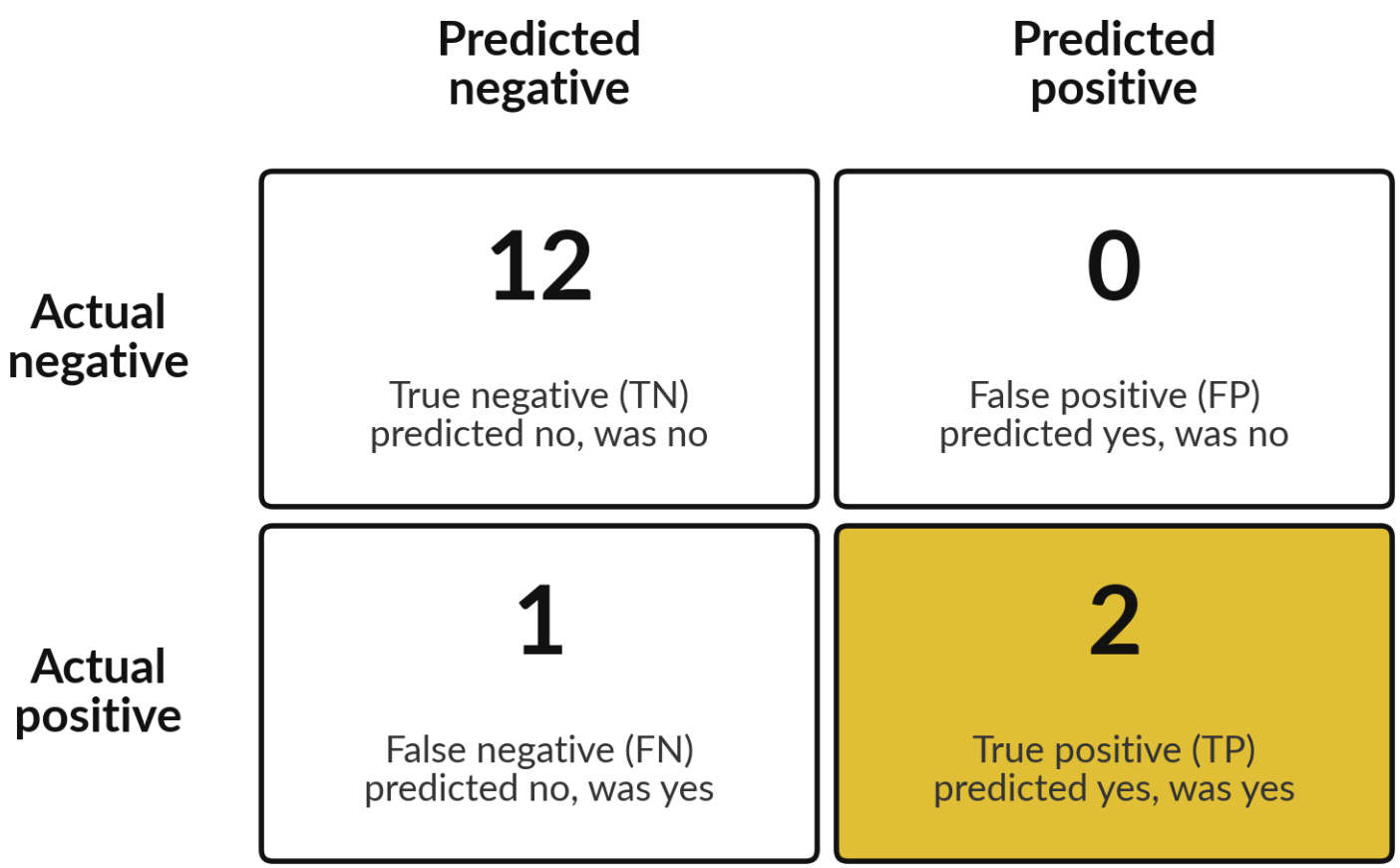
A row follows the answers down the tree to one leaf node, a yellow box. Each question is an internal node.

A leaf predicts the majority class of its rows. (n): training rows.

max_depth = 3: at most 3 questions.

1 leaf node rests on one row: a sign of overfitting.

The confusion matrix counts true and false positives and negatives



Logistic regression, decision threshold 0.5, validation set. Positive class: high cooling load.

Each of the 15 rows lands in exactly one box.

Rows: Actual class. Columns: Predicted class, as in scikit-learn.

FP is a false warning. FN is a missed positive.

Accuracy and precision are worked out from the four counts TP, FP, FN and TN

Accuracy

$$\text{accuracy} = \frac{TP + TN}{n} = \frac{2 + 12}{15} = 0.93$$

Precision

$$\text{precision} = \frac{TP}{TP + FP} = \frac{2}{2 + 0} = 1.00$$

Symbol	What it means	In our data
TP	True positive: predicted positive, actually positive	2 validation rows
FP	False positive: predicted positive, actually negative	0 rows
FN	False negative: predicted negative, actually positive	1 row
TN	True negative: predicted negative, actually negative	12 rows
n	All rows scored: TP + TN + FP + FN	15 validation rows

Counts are for logistic regression at decision threshold 0.5. The majority-class baseline has accuracy 0.80.

Recall (sensitivity) and F1: how many actual positives we find, and one blended score

Recall

$$R = \text{recall} = \frac{TP}{TP + FN} = \frac{2}{2 + 1} = 0.67$$

F1

$$F_1 = \frac{2PR}{P + R} = \frac{2 \times 1.00 \times 0.67}{1.00 + 0.67} = 0.80$$

Symbol	What it means	In our data
R	Recall (sensitivity, true positive rate): share of actual positives found	2 found of 3 actual positive rows
P	Precision: the share of the model's positive predictions that are right	1.00 (previous slide)
F1	The F1 score, a blend of P and R. It is low when either one is low	0.80
TP, FN	True positives and false negatives (previous slide)	2 and 1

Same F1 from the counts: 2TP / (2TP + FP + FN) = 4 / (4 + 0 + 1) = 0.80.

A false negative and a false positive cost different things for a building

False negative (FN)

Predicted no, but actually yes.

The cooling plant is not ready.

Rooms can get too warm.

Every FN lowers recall.

False positive (FP)

Predicted yes, but actually no.

Extra cooling starts early for nothing.

It wastes some energy.

Every FP lowers precision.

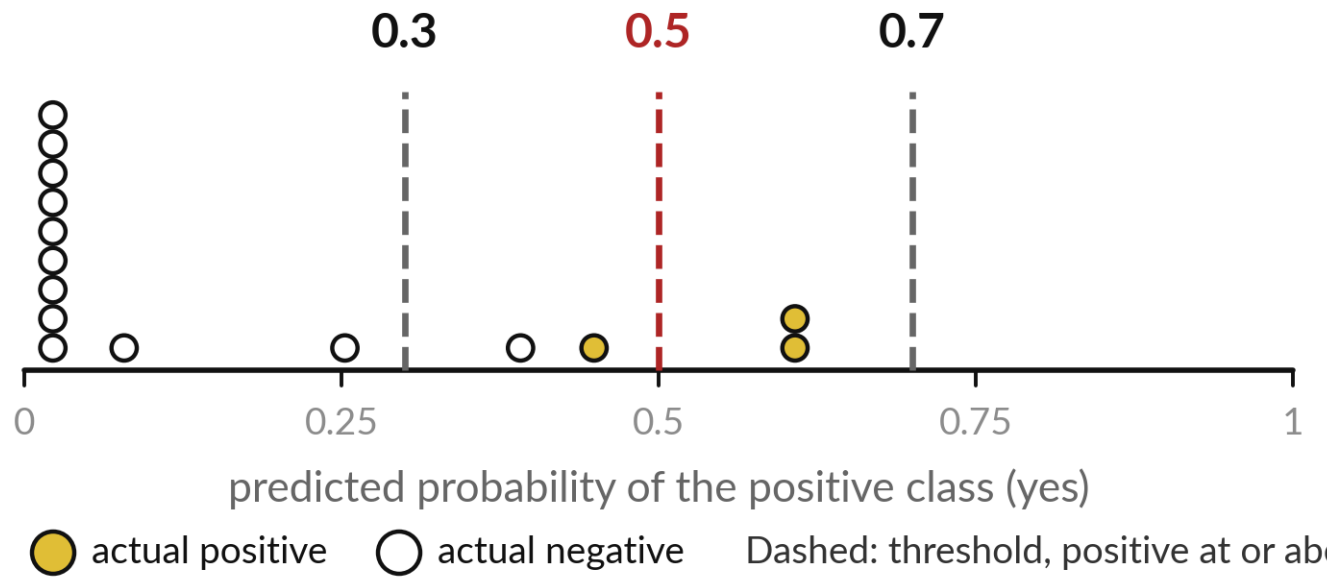
A false negative costs more here, so we watch recall.

Logistic regression and the decision tree give different counts on the same 15 rows

Measure	Logistic regression	Decision tree, depth 3	Majority-class baseline
True positives (TP)	2	2	0
False negatives (FN)	1	1	3
False positives (FP)	0	1	0
Accuracy	0.93	0.87	0.80
Precision	1.00	0.67	0.00
Recall	0.67	0.67	0.00
F1	0.80	0.67	0.00

Logistic regression has the higher F1 here (0.80 against 0.67). With 15 rows, read the gap as a hint. Precision of 0.00 for the majority-class baseline means 0 of 0: it never predicts positive.

Lowering the decision threshold never lowers recall, and it can add false positives



Threshold 0.3: TP 3, FN 0, FP 1, recall 1.00

Threshold 0.5: TP 2, FN 1, FP 0, recall 0.67

Threshold 0.7: TP 0, FN 3, FP 0, recall 0.00

A lower threshold predicts positive more often: recall rises or stays, and false positives can rise.

0.5 to 0.3: recall goes from 0.67 to 1.00; FP goes from 0 to 1.

0.5 to 0.7: recall goes from 0.67 to 0.00; FP stays at 0.

The cost of the mistakes puts a price on each one: here a false negative costs 5 and a false positive costs 1

Cost of mistakes

$$\text{cost} = \text{MISSED_COST} \times FN + \text{FALSE_ALARM_COST} \times FP$$

At threshold 0.5

$$\text{cost} = 5 \times 1 + 1 \times 0 = 5$$

Symbol	What it means	In our data
MISSED_COST	The cost of one false negative (a missed positive)	5 (invented teaching cost)
FALSE_ALARM_COST	The cost of one false positive (a false alarm)	1 (invented teaching cost)
FN, FP	False negatives and false positives at the chosen threshold	At 0.5: FN 1, FP 0
cost	The total price of the mistakes on the validation set. Lower is better	At 0.5: 5

Invented teaching costs: one missed positive costs as much as five false alarms.

The cost table picks decision threshold 0.3, the lowest cost on the validation set

	Threshold	TP	FP	FN	TN	Recall	Precision	Cost
🔔	**0.3**	3	1	0	11	1.00	0.75	5 x 0 + 1 x 1 = 1
✅	**0.5**	2	0	1	12	0.67	1.00	5 x 1 + 1 x 0 = 5
🚩	**0.7**	0	0	3	12	0.00	0.00	5 x 3 + 1 x 0 = 15

Rule: choose the threshold with the lowest cost. If two tie, take the higher threshold. At 0.7 no row is predicted positive, so precision shows 0.00.

On a larger set, a lower decision threshold finds more positives and also gives more false positives

	Decision threshold	TP (of 10 actual positive)	FP (of 90 actual negative)	True positive rate (TPR)	False positive rate (FPR)
🔔	**0.2**	10	27	1.00	0.30
✅	**0.5**	8	6	0.80	0.07
🚩	**0.8**	4	1	0.40	0.01

Illustrative example (hypothetical data), not from our data. TPR is the same as recall. Each row of this table becomes one point on a ROC curve.

Two rates give the points of the ROC curve: the false positive rate (FPR) and the true positive rate (TPR)

False positive rate (at 0.5)

$$\text{FPR} = \frac{FP}{FP + TN} = \frac{0}{0 + 12} = 0.00$$

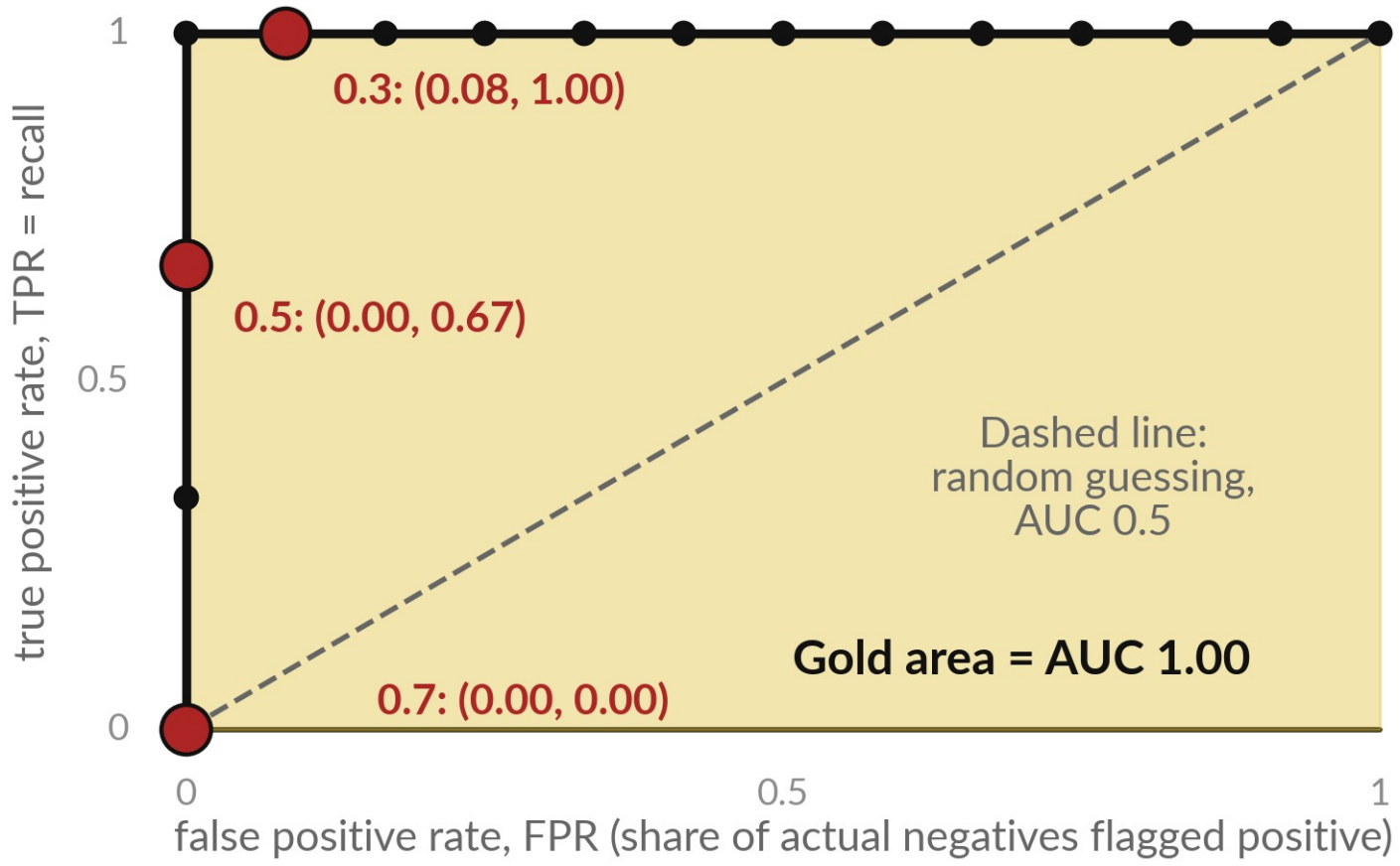
True positive rate (at 0.5)

$$\text{TPR} = \frac{TP}{TP + FN} = \frac{2}{2 + 1} = 0.67$$

Symbol	What it means	In our data
FPR	False positive rate: share of actual negatives (FP + TN) predicted positive	0 of 12 actual negative rows
TPR	True positive rate: share of actual positives (TP + FN) found. Same as recall	2 of 3 actual positive rows

Points (FPR, TPR) at thresholds 0.3, 0.5 and 0.7: (0.08, 1.00), (0.00, 0.67), (0.00, 0.00).

The ROC curve plots one point for every decision threshold: FPR across, TPR up



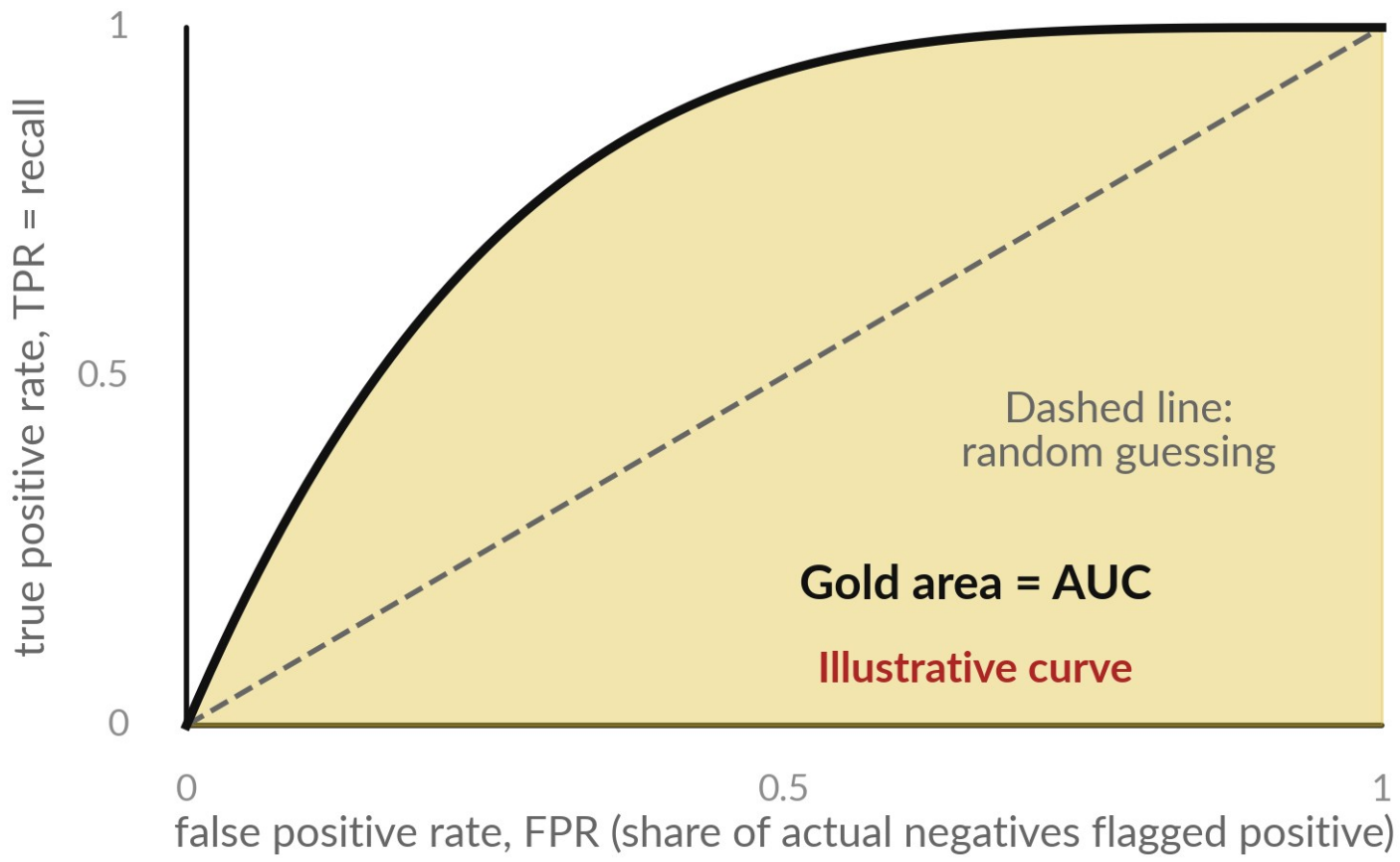
ROC is short for receiver operating characteristic. It is a graph with one point per decision threshold.

16 black points, so the line is a staircase.

Your lab plot shows only the 4 corners.

Red: thresholds 0.3, 0.5, 0.7.

AUC is the area under the ROC curve, and on a large dataset the curve is smooth



AUC (area under the curve) is the chance that an actual positive row scores higher than an actual negative row (ties count half).

Ours: 36 pairs (3 x 12), and 36 of 36 pairs are in the right order: AUC 1.00.

The curve is illustrative. With thousands of rows it looks smooth. AUC 0.5 is random guessing.

With 15 rows and 3 actual positives, every score here is shaky



The validation set has 15 rows and only 3 actual positive rows.



If one actual positive row changed its prediction, recall would move by 0.33. One changed row moves accuracy by 0.07.



These numbers teach you how the scores work. They are too few to prove which model is better.



Class 3 shows how to test a model with more care.

Summary: judge a binary classifier by counting each kind of mistake and pricing it



Make a label with a limit. Standardise the features into z-scores.



Logistic regression turns a score s into a probability with the sigmoid function.



It learns by gradient descent on the mean log loss, plus an L2 penalty in scikit-learn.



Count TP, FP, FN and TN, and work out accuracy, precision, recall and F1.



Choose the decision threshold with the lowest cost. AUC shows how well positives rank above negatives.

Read the workflow before you write it: impute, standardise, fit, predict probabilities, count

```
imputer = SimpleImputer(strategy="median")
X_train = imputer.fit_transform(
    training_data[feature_columns])
X_valid = imputer.transform(
    validation_data[feature_columns])
logistic_model = make_pipeline(
    StandardScaler(), LogisticRegression())
logistic_model.fit(X_train, y_train)
y_proba = logistic_model.predict_proba(X_valid)[:, 1]
threshold = 0.5
y_pred = (y_proba >= threshold).astype(int)
cm = confusion_matrix(y_valid, y_pred)
```

In plain words

The imputer fills each missing value (NaN) with the training median (imputation).

The pipeline chains StandardScaler and LogisticRegression.

predict_proba gives a probability per class. Column 1 is the positive class.

y_proba >= threshold gives True or False. astype(int) makes it 1 or 0: y_pred.

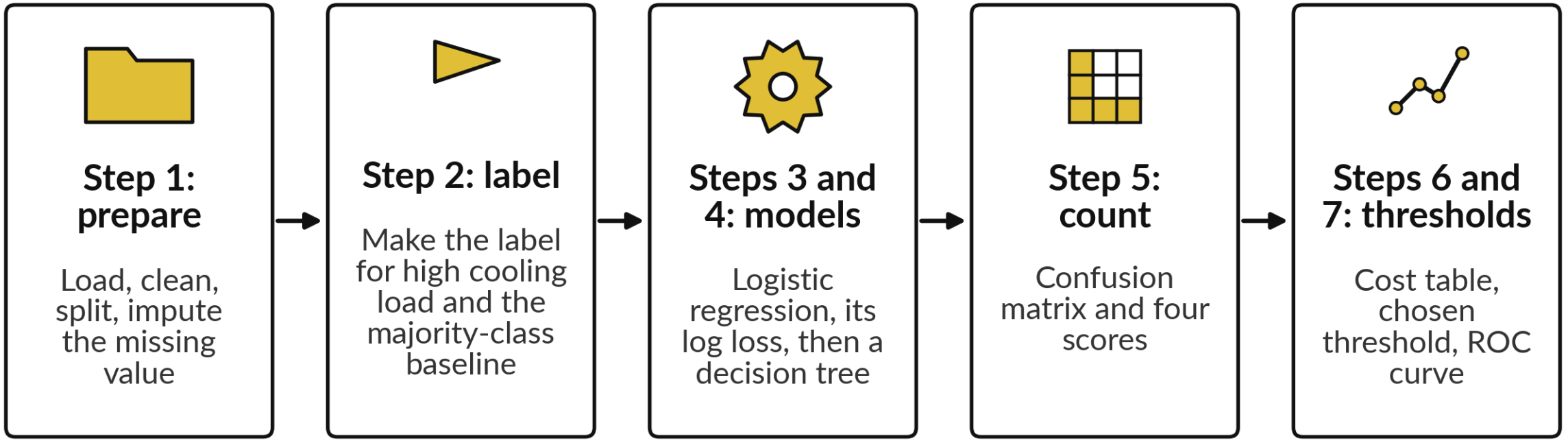
confusion_matrix puts Actual down the side and Predicted across the top, negative first: TN and FP in the top row, FN and TP below.



Part 2: the lab

Close the slides. Open the notebook. You use the same engineering_data.csv file as in Class 1.

The lab has nine steps in five stages, and each stage ends with a check



Predict first, run second, then read the output. CHANGE tasks: steps 3 and 6. Steps 8 and 9: conclusion and saving.

Check your numbers against these before you move on

What to check	You should see
Positive labels (training set, validation set); majority-class baseline accuracy	19 of 58, 3 of 15; 0.80
Learned weights (coefficients): load_pct, ambient_c, and the intercept	2.79, 0.73, -1.71
Mean log loss on the training set	0.21
Logistic regression: TP, FN, FP, TN	2, 1, 0, 12
Logistic regression: accuracy, precision, recall, F1	0.93, 1.00, 0.67, 0.80
Decision tree: TP, FN, FP, TN	2, 1, 1, 11
Cost at decision thresholds 0.3, 0.5, 0.7, and the chosen threshold	1, 5, 15; 0.3
AUC	1.00

If your number differs, stop and ask. Do not change a number to match. Find the step that went wrong.

Four messages you will meet, what they mean and what to do

	Message	What it says	What to do
	**ValueError: contains NaN**	A missing value (NaN) reached StandardScaler or the model	Run the imputation step (SimpleImputer with the training median)
	**Classification metrics can't handle a mix of binary and continuous targets**	You gave probabilities where a predicted class (0 or 1) was needed	Compare y_proba with the threshold first, to get y_pred
	**ConvergenceWarning**	The solver stopped before it reached the bottom of the loss. The cell still runs	Standardise the features, or raise max_iter
	**NameError**	A cell above has not been run yet	Run all cells from the top, in order

How this class counts towards your coursework



Block 1 is Class 1 and Class 2. It is practice: it does not count toward your final mark.



Assignment part A (A7, regression) repeats the Class 1 lab. Part B (A8, classification and cost) repeats today's lab.



The Block 1 assignment closes on Tuesday 17 November, 11:00 (Kampala time).



The Block 1 quiz is two online forms, Q7 (Class 1) and Q8 (Class 2). They open in the last 20 minutes today, each for 24 hours.

Save the notebook and keep your cost table

What you keep from today

Your lab notebook, saved with all cells run in order, and the confusion matrix and the cost table from the lab.

Our question today

Can a binary classifier say yes or no to "high cooling load", how does it learn, and how do we count and price its mistakes?

Next class

Class 3: can I trust this model? We test it more carefully.

Take away

Saved notebook
Restart and run all, then save

Confusion matrix
TP, FN, FP and TN

Cost table
Thresholds 0.3, 0.5 and 0.7

Bring next time
The same laptop and notebook